

Prueba Técnica Automatización (QA)

1. Objetivo

Evaluar la capacidad técnica del candidato para diseñar, estructurar e implementar un framework de pruebas robusto, resiliente y mantenible bajo el patrón **Page Object Model (POM)**. La prueba exige la cobertura completa de flujos críticos sobre una aplicación híbrida, resolviendo problemas típicos de sincronización y validaciones complejas de interfaz mediante el uso combinado de automatización basada en selectores y técnicas de reconocimiento visual (OCR y OpenCV).

2. Contexto de la Aplicación bajo Prueba (AUT)

- **Aplicación Target:** DemoBank — aplicación Android nativa compilada de forma independiente (standalone).
- **Estado:** App 100% mockeada, sin conectividad a un backend real.
- **Flujos Navegacionales:** Splash Screen → Login → Home (Tabs: Home / Movimientos) → Transferir (Modal de 3 pasos) → Pagar Servicios (Modal de 3 pasos).
- **Credenciales de Prueba:** demo@demo.com / 1234
- **Cuentas Mockeadas:** Cuenta Corriente (\$1,500,000.00) y Cuenta Ahorros (\$955,450.00).

REQUISITO TÉCNICO CRÍTICO:

La automatización debe apuntar estrictamente al APK(DemoBank) standalone.



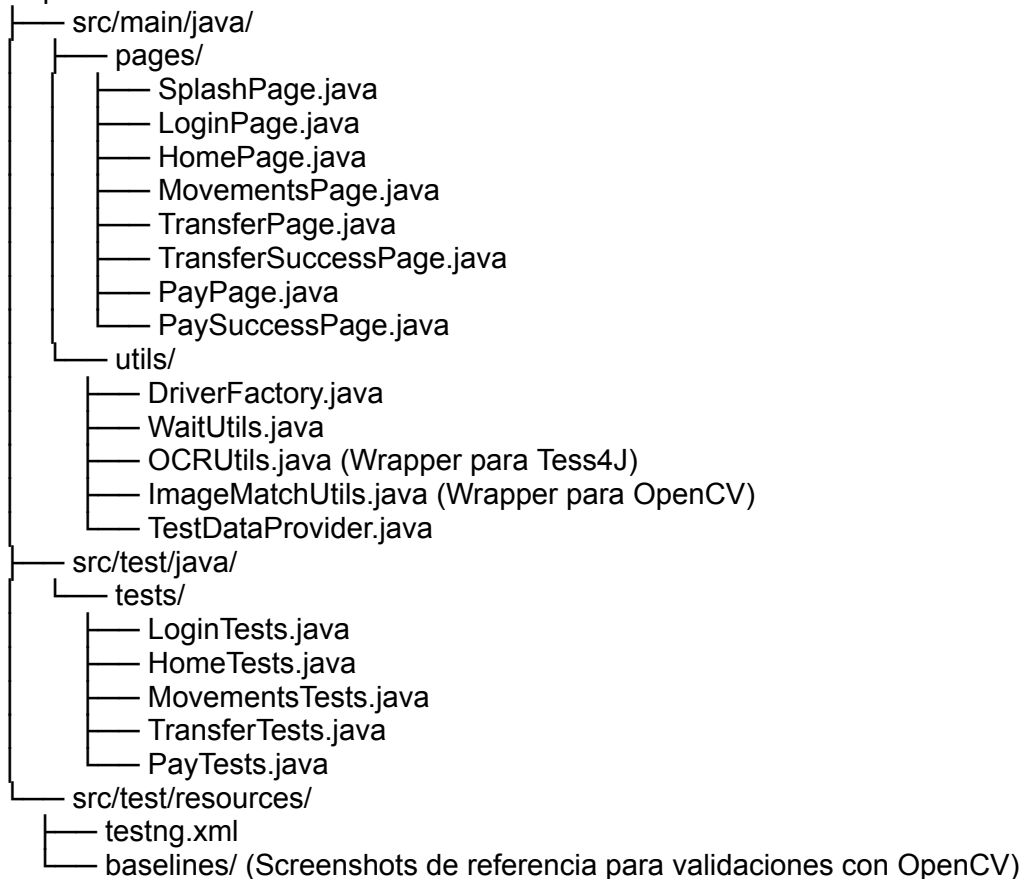
descargar aplicacion demoBank

3. Stack Tecnológico Obligatorio

- **Lenguaje Core:** Java 11+
- **Gestor de Dependencias:** Maven (archivo pom.xml estructurado).
- **Motores de Automatización:** Appium Server 2.x + Appium Java Client (Última versión estable) y Selenium WebDriver como API base subyacente.
- **Mapeo de Selectores:** Appium Inspector (Uso obligado para inspeccionar el árbol de accesibilidad antes de codificar).
- **Test Runner:** TestNG (Uso de anotaciones de ciclo de vida @BeforeSuite/@BeforeMethod para el setup del driver y DataProviders para pruebas guiadas por datos).
- **Reconocimiento de Texto:** Tesseract OCR (vía **Tess4J**) para la lectura de montos o textos en elementos complejos sin identificadores de accesibilidad fiables (ej. el número de saldo sobre la tarjeta con gradiente).
- **Visión Computacional:** OpenCV (vía bindings Java o **JavaCV**) para realizar comparaciones visuales píxel a píxel contra imágenes de referencia (baselines) con umbrales de similitud tolerantes a regresiones estéticas.

4. Estructura del Proyecto Esperada

El código fuente del framework debe respetar la siguiente distribución de paquetes y carpetas:



5. Casos de Prueba Mínimos Exigidos

Se deben automatizar un **mínimo de 15 casos de prueba funcionales** distribuidos estratégicamente a través de los siguientes flujos funcionales (incluyendo caminos positivos, escenarios negativos y condiciones de borde):

Módulo 1: Login

- **Autenticación Exitosa:** Ingreso con credenciales válidas y redirección inmediata al Home de la aplicación.
- **Validación de Email Vacío:** Dejar el campo de correo vacío, validar que se despliegue el mensaje de error de validación y que se bloquee la navegación.
- **Validación de Contraseña Vacía:** Dejar el campo de password vacío, verificar mensaje de error y bloqueo de flujo.
- **Funcionalidad del Toggle de Contraseña:** Validar el funcionamiento del botón para mostrar/ocultar el texto de la contraseña, certificando el cambio de propiedad del elemento en la interfaz móvil.

Módulo 2: Home

- **Consistencia del Saldo Consolidado:** Validar que el saldo total visible en pantalla equivalga exactamente a la suma de las cuentas individuales (Corriente y Ahorros). *Esta validación del número renderizado debe realizarse obligatoriamente vía OCR debido a que es un componente gráfico personalizado sin testID expuesto.*
- **Interactividad de Cuentas:** Cambiar entre las pestañas de las cuentas disponibles y certificar que el número de cuenta y el saldo se actualizan dinámicamente en pantalla.
- **Navegabilidad de Accesos Rápidos:** Validar que los botones de acceso directo para *Transferir, Pagar y Movimientos* redirigen a sus pantallas correspondientes de forma satisfactoria.
- **Cierre de Sesión Seguro:** Ejecutar la acción de Logout y verificar que la app destruye la vista actual y retorna al usuario a la pantalla de Login.

Módulo 3: Movimientos

- **Buscador de Transacciones (Filtro Parcial):** Digitar un criterio de búsqueda de texto parcial y validar que el filtro responda de forma correcta ignorando mayúsculas y minúsculas (case-insensitive).
- **Filtro Avanzado de Ingresos:** Activar el filtro de "Ingresos" y validar programáticamente que todos los movimientos listados posean valores monetarios positivos.
- **Filtro Avanzado de Gastos:** Activar el filtro de "Gastos" y verificar que todos los movimientos listados correspondan a montos negativos.
- **Manejo de Estado Vacío (Empty State):** Realizar una búsqueda aleatoria sin coincidencias y certificar la aparición de la ilustración o texto informativo de "Sin Resultados".

Módulo 4: Transferencias

- **Flujo Feliz de Envío de Dinero:** Selección de contacto → ingreso de monto válido → confirmación de transacción → visualización de pantalla de éxito con el nombre y monto correctos.
- **Validación de Saldo Insuficiente:** Intentar transferir un monto superior al saldo disponible de la cuenta origen y comprobar la aparición del mensaje "Saldo insuficiente" junto con el bloqueo del botón continuar.
- **Validación de Monto Inválido:** Intentar avanzar con un monto vacío o equivalente a cero, verificando el mensaje de error "Ingresa un monto válido".
- **Impacto en Saldo Origen (Control Pre/Post):** Validar que el saldo de la cuenta origen se descuente de manera exacta inmediatamente después de procesarse la transferencia. *Obligatorio realizar la lectura de los montos antes y después de la operación mediante OCR.*
- **Auditoría en el Historial:** Verificar que la nueva transferencia se registre de forma correcta en el módulo de Movimientos con su respectivo título y valor debitado.

Módulo 5: Pagos de Servicios

- **Flujo Feliz de Pago de Servicios Públicos:** Completar de forma exitosa el flujo de pago para al menos dos (2) empresas prestadoras de servicios distintas.
- **Precarga Dinámica de Monto:** Validar que al seleccionar el tipo de servicio a pagar, la aplicación precargue automáticamente en el formulario el monto sugerido o facturado del mes.
- **Controles de Fondos y Montos:** Replicar las validaciones estrictas de saldo insuficiente y montos inválidos aplicadas en el módulo de transferencias.
- **Auditoría de Categorización:** Certificar que el pago procesado aparezca reflejado en el listado de Movimientos bajo la categoría explícita de "Servicios".

6. Reglas de Uso de Inteligencia Visual (OCR / OpenCV)

- **Aserciones mediante Tesseract OCR (Mínimo 3):** El framework debe implementar Tess4J para leer texto o cifras monetarias en al menos tres (3) validaciones donde los localizadores nativos no sean confiables (ej. saldo de tarjetas con fondos con gradientes, estados dinámicos). El candidato deberá justificar técnicamente en el reporte por qué recurrió al OCR en cada caso específico.
- **Validación Visual mediante OpenCV (Mínimo 1):** Se debe ejecutar una comprobación de regresión estética comparando un screenshot en tiempo real contra una imagen base de referencia almacenada en `src/test/resources/baselines/`. La prueba se considerará aprobada si el algoritmo de correlación visual arroja un umbral de similitud (Match Score) igual o superior al 95%.

7. Lineamientos de Calidad y Criterios de Evaluación

1. **Políticas de Sincronización:** Uso estricto de esperas explícitas (`WebDriverWait`). Queda terminantemente prohibida la inclusión de `Thread.sleep()` en los scripts, salvo justificaciones extremadamente puntuales asociadas al ciclo de vida de animaciones

- de React Native que estén documentadas en el código.
2. **Arquitectura Limpia (POM):** Separación absoluta de responsabilidades. Las clases del paquete pages encapsularán únicamente localizadores y métodos interactivos; bajo ninguna circunstancia contendrán aserciones (Asserts de TestNG). Las aserciones pertenecen exclusivamente a la capa de tests.
 3. **Externalización de Datos:** Todos los valores de prueba fijos, credenciales y montos lógicos deben extraerse a través del TestDataProvider, evitando el hardcodeo de datos en los métodos de prueba.
 4. **Resiliencia ante Flakiness de React Native:** El framework debe implementar al menos un mecanismo de reintento en caso de fallas por transiciones de UI (ej. interceptores de TestNG o lógica de reintento controlada en esperas fluidas) debido a la naturaleza asíncrona de requestAnimationFrame de React Native.

8. Entregables Esperados

1. **Código Fuente del Proyecto:** Proyecto Maven completo y libre de errores de compilación, listo para ser ejecutado mediante el comando de consola estándar mvn test.
2. **Reporte Detallado de Ejecución:** Archivo HTML autogenerado mediante **ExtentReports** o **Allure**, configurado de tal manera que anexe automáticamente capturas de pantalla únicamente ante la ocurrencia de fallas en los casos de prueba.
3. **Documento README.md Técnico:** Archivo guía con los pasos claros para el setup del entorno, indicando versión de Appium Server, configuración de Appium Inspector, emulador de Android configurado y pasos para la descarga e instalación manual de la APK dummy.

Importante: Se validará el uso de modelos de IA en el desarrollo de la prueba.